

Exposing Per-Pod OverlayFS Amplification in Kubernetes via eBPF: Whiteout Metadata Churn and Copy-Up Inode Cost

Anonymous Author

Abstract—In Kubernetes clusters, pods run inside containers whose root filesystems are managed by containerd using OverlayFS as its default snapshotter. The copy-up and whiteout mechanisms of this union filesystem introduce hidden per-pod filesystem overheads that are invisible to conventional node-level monitoring tools, leaving cluster administrators unable to attribute resource pressure to individual pods. This paper presents a dual-layer eBPF framework that simultaneously traces VFS and OverlayFS kernel operations, maps events to Kubernetes pods via cgroup identity, and exports per-pod filesystem metrics through Prometheus. Controlled evaluation on a MicroK8s cluster reveals two key findings: lower-layer file deletions produce a consistent 3:1 VFS metadata amplification factor due to whiteout handling, while copy-up operations exhibit a 1:1 inode consumption ratio that remains constant regardless of container image layer depth. Comparative analysis confirms that neither standard Kubernetes monitoring tools nor state-of-the-art eBPF security tracers provide equivalent per-pod semantic visibility. The framework operates at under 11 millicores CPU overhead, confirming production viability.

Index Terms—eBPF, container orchestration, Linux filesystem, storage resource management, cloud-native observability

I. INTRODUCTION

In Kubernetes, each pod's root filesystem is assembled by containerd, the default container runtime using OverlayFS as its snapshotter. OverlayFS stacks read-only image layers beneath a writable upper layer, enabling efficient image sharing across pods on the same node. However, this architecture introduces distinct kernel-level filesystem behaviors that affect how resources are consumed on the host. File modifications trigger copy-up operations, which duplicate lower-layer files into the upper layer and allocate new inodes, while deletions of lower-layer files are handled through whiteout mechanisms that introduce additional metadata operations in the upper layer. As pod density increases, the cumulative effects of these behaviors, particularly inode consumption from copy-up and sustained metadata activity from whiteouts can contribute to filesystem resource pressure, potentially leading to inode exhaustion and degraded system performance.

Despite the operational impact of these behaviors, existing monitoring tools cannot attribute filesystem resource usage to individual pods. Standard tools such as cAdvisor and Prometheus Node Exporter provide node-level filesystem metrics but lack per-pod granularity and have no visibility into OverlayFS-internal kernel operations. eBPF-based security tracers observe filesystem activity at the VFS syscall boundary

but cannot distinguish the semantic origin of individual VFS operations, such as whether they are triggered by copy-up or whiteout handling. Consequently, cluster administrators have no mechanism to identify which pods are responsible for inode consumption, which are generating metadata-intensive workloads, or how these behaviors contribute to overall filesystem pressure.

This paper makes the following contributions:

- A dual-layer eBPF framework that simultaneously traces both VFS-layer and OverlayFS-layer kernel operations, maps events to Kubernetes pods via cgroup identity, and exports per-pod filesystem metrics through Prometheus.
- Empirically quantifies for the first time that lower-layer file deletions produce a consistent 3 : 1 VFS metadata amplification factor associated with whiteout handling.
- Empirically establishes that copy-up operations consume inodes at a precise 1 : 1 ratio per operation.
- Demonstrates through comparative evaluation that no existing monitoring tool provides equivalent per-pod OverlayFS semantic visibility.

The remainder of this paper is organized as follows. Section II describes the framework design and experimental setup. Section ?? presents the evaluation results. Section ?? discusses the findings and their implications. Section ?? concludes the paper.

II. SYSTEM DESIGN

The proposed framework consists of two cooperating components: a kernel-resident eBPF tracing module written in C and a user-space aggregation daemon written in Go. The framework is deployed as a privileged DaemonSet on each Kubernetes worker node. Fig. 1 illustrates the end-to-end architecture. The framework distinguishes between two categories of filesystem impact: inode consumption, primarily driven by copy-up operations, and metadata amplification, associated with whiteout handling and other OverlayFS internal behaviors.

A. Dual-Layer eBPF Probe Architecture

The eBPF module attaches probes at two distinct kernel layers to capture both inode-affecting VFS operations and OverlayFS semantic behaviors.

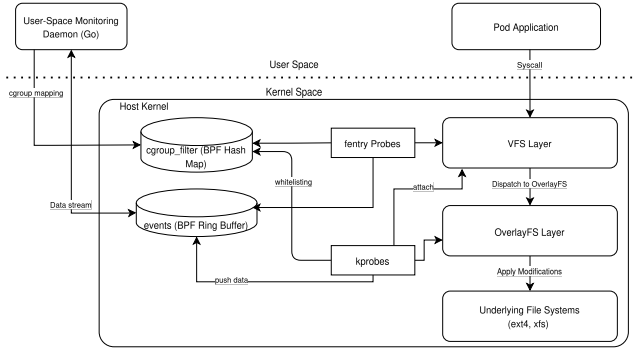


Fig. 1. End-to-end framework architecture.

1) *VFS Layer (fentry Probes)*: Function-entry (fentry) probes are attached to core Virtual File System routines using BPF Type Format (BTF) metadata. The fentry mechanism patches the first instruction of the target function to redirect execution into the eBPF handler with near-zero overhead (no `int3` interrupt or stack-frame copying), receiving fully typed function arguments. Table I lists the traced VFS functions and their inode impact.

TABLE I
VFS-LAYER PROBES ATTACHED VIA FENTRY.

Kernel Function	Probe	Δ Inode	Purpose
<code>vfs_create</code>	fentry	+1	File creation
<code>vfs_mkdir</code>	fentry	+1	Directory creation
<code>vfs_symlink</code>	fentry	+1	Symlink creation
<code>vfs_unlink</code>	fentry	-1	File deletion
<code>vfs_rmdir</code>	fentry	-1	Directory removal
<code>vfs_rename</code>	fentry	0	Rename (no new inode)
<code>vfs_link</code>	fentry	0	Hard link (no new inode)
<code>vfs_mknod</code>	kprobe	+1	Device node creation

2) *OverlayFS Layer (kprobes)*: Kernel probes (kprobes) are bound to internal OverlayFS routines, which are not always BTF-annotated and whose symbol names may vary across kernel versions. The daemon iterates through a candidate symbol list per probe and attaches to the first resolved match, ensuring cross-version compatibility. Table II lists the traced OverlayFS functions.

TABLE II
OVERLAYFS-LAYER PROBES ATTACHED VIA KPROBE.

Kernel Function(s)	Δ Inode	Purpose
<code>ovl_copy_up_one / ovl_copy_up_start</code>	+1	Copy-up: lower to upper layer
<code>ovl_unlink / ovl_do_unlink / ovl_remove</code>	-1	OverlayFS-level file deletion
<code>ovl_cleanup_and_whiteout</code>	0	Whiteout handling
<code>ovl_lookup</code>	0	Layer traversal resolution

B. Cgroup-Based Event Filtering

Every probe begins execution with a two-stage filtering sequence. First, the BPF helper `bpf_get_current_cgroup_id()` retrieves the cgroup v2 identifier of the process triggering the syscall. This

identifier is looked up in a BPF hash map (`cgroup_filter`) that serves as a whitelist of monitored container cgroups, populated and periodically refreshed by the user-space daemon. If the calling process does not belong to a monitored pod, the probe returns immediately with negligible overhead. Second, if the cgroup is whitelisted, a slot is reserved in a lock-free BPF ring buffer (4 MB capacity). The probe then populates a fixed-size event structure comprising the operation type identifier, the cgroup ID, and an inode-delta value (+1, -1, or 0) before asynchronously submitting the event to user space. The ring buffer was chosen over the legacy `PERF_EVENT_ARRAY` for its superior memory efficiency and absence of per-CPU buffer fragmentation.

C. User-Space Daemon and Metric Export

The Go-based daemon executes four principal functions:

(1) **eBPF Lifecycle Management**. The daemon loads the pre-compiled eBPF object into the kernel, validates it via the kernel verifier, and attaches all probes. For fentry probes, BTF-based binding is used; for kprobes, the daemon tests multiple candidate symbol names per function to ensure compatibility across kernel versions.

(2) **Cgroup-to-Pod Resolution**. Two concurrent mapping tables translate raw kernel cgroup IDs into Kubernetes metadata. A *cgroup inode map* is populated by periodically walking `/sys/fs/cgroup/kubepods.slice` and extracting the filesystem inode number of each cgroup directory (identical to the cgroup IDs reported in kernel space). A *pod identity map* is maintained via a persistent Kubernetes API watch stream, mapping each pod's UID to its display name and namespace. Synchronization occurs on a 5-second interval, with a fallback full API re-query for newly scheduled pods.

(3) **Event Consumption Pipeline**. Events from the BPF ring buffer are consumed into a high-capacity Go channel (1 M elements). Each event undergoes binary decoding, cgroup path resolution, pod UID extraction, and pod metadata association, producing a fully attributed event record.

(4) **Prometheus Metric Exportation**. Three metrics are exported via an HTTP endpoint, as summarized in Table III. Upon pod deletion, associated label sets are cleaned up to prevent stale time series.

TABLE III
SUMMARY OF PROMETHEUS METRICS EXPOSED BY THE FRAMEWORK, INCLUDING THEIR TYPES AND PURPOSES.

Metric	Type	Purpose
<code>container_inode_operation_total</code>	Counter	Per-pod, per-operation cumulative count
<code>container_inode_net_delta</code>	Gauge	Running net inode Δ per pod
<code>node_inode_free</code>	Gauge	Host free inode count

While inode-related metrics capture direct resource consumption, OverlayFS semantic events are additionally used

to derive metadata amplification characteristics at the per-pod level.

D. Metadata Amplification Factor Definition

To quantify the hidden kernel-level cost of OverlayFS semantic operations, the framework computes a Metadata Amplification Factor (MAF) defined as the ratio of physical VFS-layer kernel operations to user-initiated OverlayFS semantic operations within a given time window:

$$\text{MAF} = \frac{\sum \text{irate}(\text{vfs_.*})}{\sum \text{irate}(\text{ovl_unlink} | \text{ovl_copy_up})} \quad (1)$$

The denominator counts user-initiated OverlayFS semantic operations: `ovl_copy_up` events represent file modifications that trigger copy-up, and `ovl_unlink` events represent user-space file deletions. Kernel-internal consequences of these operations such as whiteout-related operations are intentionally classified as amplification overhead in the numerator rather than as base operations, as they represent hidden kernel cost invisible to the container process. An MAF of 1 indicates a one-to-one relationship between semantic operations and physical VFS events; a value of 3 indicates that each user-space operation generates three physical kernel operations. The MAF is computed per-pod using the Prometheus `irate()` function over a 5-minute sliding window and exported as a derived metric alongside the three primary metrics in Table III. This behavior highlights that metadata amplification and inode consumption are driven by different operation types, with copy-up dominating inode growth and whiteout handling contributing primarily to metadata overhead.

E. Comparative Experimental Setup

To empirically validate the observability gap, four existing monitoring tools were deployed concurrently alongside the proposed framework on the same MicroK8s worker node: cAdvisor (Kubelet-integrated, representing native Kubernetes container metrics), Prometheus Node Exporter v1.10.2 (DaemonSet, representing standard node-level infrastructure monitoring), Aqua Tracee v0.24.1 (Helm Chart, representing state-of-the-art eBPF runtime security tracing), and Sysdig Falco v0.43.0 (Helm Chart, representing rule-based eBPF syscall monitoring). Each tool operated under its default production configuration with no custom rules or filters applied. The synthetic workloads defined in Section II were executed sequentially with all five tools active simultaneously, ensuring identical kernel states and workload conditions. A capability was recorded as detected if the tool natively produced a metric or event that could be directly attributed to the specific OverlayFS operation without requiring external post-processing or manual correlation.

III. RESULTS

The framework was evaluated on a MicroK8s cluster deployed inside a KVM environment using six controlled synthetic workloads, each engineered to isolate a specific OverlayFS kernel behavior. The eBPF tracer daemon was

active throughout all workloads, with four existing monitoring tools deployed concurrently on the same worker node under identical conditions.

A. Whiteout Metadata Amplification and Inode Masking

The *whiteout-test-pod* executed 10,000 sequential deletions of lower-layer files using a custom image (~2-layer stack). The framework captured `ovl_whiteout` and `vfs_link` activity at 84.4–86.4 ops/sec throughout the 105-second workload window. The Metadata Amplification Factor (MAF) held a steady value of **3:1** throughout, decomposing as three distinct physical VFS operations per user-space deletion: an upper-layer dentry removal (`vfs_unlink`), a hard-link installation (`vfs_link`) to bind the shared whiteout character device into position, and an atomic rename (`vfs_rename`) to place the marker in the upper directory. The whiteout character device itself is created once via `vfs_mknod` (observed delta: 1) and reused across all 10,000 deletions through hardlinking, a kernel optimization explaining the near-neutral node-level inode impact. The `ovl_lookup` panel confirmed layer traversal at a matching rate of ~86 ops/sec, indicating that each deletion required a full layer traversal to locate the target file before placing the whiteout.

The Node Health Overview panel showed **near-neutral** change in free inodes despite 10,000 deletions completing. The Net Inode Delta panel recorded $-7,399$ for `ovl_unlink` operations, yet the node-level free inode metric showed no corresponding recovery. This confirms the deceptive masking effect: although lower-layer deletions trigger multiple kernel-level metadata operations, the node-level free inode metric shows near-neutral change. This indicates that whiteout handling introduces significant metadata activity without producing a proportional change in observable inode availability. An administrator monitoring only `node_inode_free` would therefore observe no pressure signal during a bulk lower-layer deletion burst, despite substantial kernel activity occurring beneath the surface.

The workload was repeated against a custom deep layer image (~155 layers). Operation rates elevated to 153–162 ops/sec and the workload completed in 45 seconds. The MAF remained **3:1** identically, confirming depth-independence.

B. Copy-Up Inode Cost and Depth Independence

The *copyup-test-pod* appended data to 10,000 existing lower-layer files, forcing a copy-up for each modification. The framework recorded `ovl_copy_up` and `vfs_link` activity at 61.8–64.8 ops/sec across the 135-second workload. The MAF registered a consistent **1:1** ratio. The Net Inode Delta panel accumulated exactly **+10,000** inodes attributed exclusively to `ovl_copy_up`, and the Node Health Overview panel confirmed a corresponding decline of precisely **10,000 free inodes** on the host disk, establishing the complete causal chain: one append → one `ovl_copy_up` → one `vfs_link` → one host inode consumed. The `ovl_lookup` panel confirmed per-operation layer traversal at the same rate.

The identical workload executed against the 155-layer deep image produced rates of 76.8–95.2 ops/sec with an identical **10,000-inode** host-level decline and MAF of **1:1**. Table IV summarizes all four depth-variant workloads.

TABLE IV
SHALLOW VS. DEEP STACK: KEY METRICS COMPARISON

Metric	S-WO	D-WO	S-CU	D-CU
Peak rate (ops/s)	84	155–160	63.4	80–90
MAF	3:1	3:1	1:1	1:1
Inode impact	~0	~0	–10K	–10K
Duration (s)	105	45	135	90

S = Shallow, D = Deep, WO = Whiteout, CU = Copy-Up

The MAF remains structurally consistent regardless of layer depth, confirming that amplification is an intrinsic property of the operation type rather than image architecture. Deep-stack workloads completed faster at higher burst rates, reflecting per-operation traversal latency through the 155-layer stack compressing the scheduling window. The `ovl_lookup` rate matched the primary operation rate in all four scenarios, quantifying the hidden per-operation traversal cost that VFS-only tracers cannot attribute to a specific container.

C. Mixed Workload and Dynamic Amplification

A three-phase workload executed, consisting of 1,000 lower-layer file modifications (Phase 1), 500 lower-layer deletions (Phase 2), and 1,000 new upper-layer file creations (Phase 3), completing within a ~45-second active window.

The framework produced temporally distinct signatures across all phases. In Phase 1, `ovl_copy_up` and `vfs_link` activity was recorded at ~63.2 ops/sec with MAF at **1.0**. In Phase 2, the panel signatures shifted entirely to `ovl_whiteout` and `vfs_unlink` at ~35.8 ops/sec, and the MAF rose to **1.8**. In Phase 3, `vfs_rename` activity was recorded for new upper-layer creations with no corresponding `ovl_copy_up` events, confirming that direct upper-layer creations do not trigger copy-up. The MAF settled at a steady **1.5** as a weighted average.

The Net Inode Delta tracked: +2,000 for `ovl_copy_up`, +1 for `vfs_mkdir`, and –500 for `ovl_unlink`. The Node Health Overview confirmed a total decline of **1,563 free inodes** significantly more than the deletion count alone would suggest, confirming the hidden compounding cost of mixed workloads. A single-layer VFS-only monitor would have reported undifferentiated filesystem events throughout, unable to distinguish a copy-up-triggered `vfs_link` in Phase 1 from a whiteout-triggered `vfs_link` in Phase 2. The dynamic MAF serves as a real-time proxy for workload characterization: a rising factor signals whiteout-heavy activity while a value near 1.0 indicates copy-up or direct-creation dominated workloads.

D. Framework Performance Overhead

CPU and memory footprint was recorded during the two highest-stress workloads. Table V summarizes the results.

Peak CPU remained under **11 millicores** (~1.0% of a single core) across both scenarios. The 4 MB BPF ring buffer

TABLE V
FRAMEWORK PERFORMANCE OVERHEAD UNDER PEAK STRESS

Workload	CPU (mc)	Mem (MB)
Evasion burst (8,000 events)	1.47	10.2
Deep whiteout (155–160 ops/s, 155 layers)	10.06	3.0

mc = millicores

and 1,000,000-element Go ingestion channel produced **zero dropped events** in both cases. The minimal overhead is attributable to in-kernel cgroup filtering, which discards all non-containerized host events before they cross the kernel-user boundary, and the lock-free BPF ring buffer, which eliminates synchronization overhead between the kernel producer and the Go consumer.

E. Comparative Analysis Against Existing Tools

Table VI summarizes the observability capabilities of all five tools evaluated during the *metaamp-test-pod* mixed workload.

TABLE VI
OBSERVABILITY CAPABILITY COMPARISON

Capability	Ours	cAdv.	N.Exp.	Tracee	Falco
Copy-up detected	✓	×	×	×	×
Whiteout detected	✓	×	×	×	×
Per-pod inode delta	✓	~	×	×	×
Semantic layer vis.	✓	×	×	×	×
Node free inodes	✓	×	✓	×	×
Per-pod attribution	✓	~	×	~	~
Total writes	FS ✓	✓	✓†	✓	✓

~ = partial † = host-level only

N.Exp. = Node Exporter, cAdv. = cAdvisor

cAdvisor provided per-container byte-level I/O throughput via `container_fs_writes_bytes_total` but cannot distinguish a direct upper-layer write from a copy-up-triggered modification. Both operations appear as identical byte counter increments with no indication that one consumed an additional host inode. Node Exporter confirmed the node-level inode decline of ~1,501 during the mixed workload but provides exclusively host-level metrics with no pod boundary awareness and no OverlayFS semantic visibility.

Aqua Tracee and Sysdig Falco represent the current state of the art in eBPF-based container tracing and successfully attributed raw file access events to specific pods via container engine enrichment. However, both tools operate exclusively at the VFS syscall boundary and are architecturally incapable of observing OverlayFS-internal kernel routines. When the three-phase workload executed 500 lower-layer deletions in

Phase 2, both tools reported 500 `unlinkat` syscalls, an action that from the syscall boundary appears to release inodes. Neither tool had any awareness that OverlayFS intercepted those deletions and silently created whiteout-related metadata entries in the upper layer, introducing additional kernel-level operations without a corresponding, clearly observable inode impact at the node level. The 3:1 amplification factor and the near-neutral node-level inode impact were entirely invisible to both tools. Similarly, Phase 1 `ovl_copy_up` events were reported only as `openat` syscalls with no visibility into the copy-up mechanism, the resulting `vfs_link`, or the one-to-one inode consumption.

The proposed framework was the only tool evaluated capable of illuminating the complete causal chain: which pod, performing which physical VFS operation, triggered which OverlayFS semantic behavior, resulting in an exact quantifiable inode impact on the host node. This gap is not an implementation deficiency in the existing tools. It is a structural consequence of where they attach to the kernel. Closing it requires probes placed at the OverlayFS layer itself, which is the core architectural contribution of the dual-layer design.

IV. DISCUSSION

The empirical results establish three findings that collectively reframe how filesystem resource pressure (inode consumption and metadata amplification) should be understood and monitored in OverlayFS-based Kubernetes deployments.

A. Amplification is Operation-Dependent, Not Depth-Dependent

The most practically significant finding is that filesystem amplification behavior is determined entirely by the type of filesystem operation rather than container image architecture. Whiteout operations consistently produced a 3:1 MAF regardless of whether the image had 2 layers or 155 layers, and copy-up operations consistently produced a 1:1 MAF under the same conditions. This depth-independence contradicts the common practitioner assumption that reducing image layer count will reduce filesystem resource pressure (inode consumption and metadata amplification) from file modifications or deletions. The data shows that layer count affects traversal latency and burst rate, but not the per-operation inode cost. An administrator optimizing image architecture to reduce layer depth will see improved pull performance and lower `ovl_lookup` overhead, but will not reduce inode consumption from copy-up operations, nor reduce metadata amplification associated with whiteout handling. This distinction is only observable through a framework that traces both the VFS and OverlayFS kernel layers simultaneously, a VFS-only or node-level monitor cannot separate operation-type cost from layer-depth cost.

B. Whiteout Deletions Are Deceptively Invisible

The 3:1 whiteout amplification finding reveals a structural blind spot in conventional monitoring with direct operational

consequences. When a pod performs bulk lower-layer deletions, the kernel generates whiteout-related metadata operations to maintain OverlayFS consistency in the upper layer. These operations introduce additional VFS activity that is not reflected in node-level inode availability metrics, rather than allocating a new inode per deletion. Consequently, an administrator monitoring `node_inode_free` via Node Exporter observes a near-neutral inode signal during this activity, potentially providing false confidence that the deletion workload is operationally benign.

In reality, three physical VFS operations, a `vfs_unlink` to remove the upper-layer entry, a `vfs_link` to hardlink the shared whiteout, and an atomic `vfs_rename` to place it, are executed per user-space deletion, each contributing to metadata churn on the host filesystem. Because the lower-layer files themselves are read-only and persist on disk regardless of the overlay deletion, no corresponding increase in observable free inodes is visible at the node level.

This hidden overhead becomes particularly significant in high-density clusters where multiple pods execute deletion-heavy workloads concurrently. The compounding VFS operations impose sustained metadata I/O pressure and directory-entry bloat on the upper-layer filesystem, degrading performance without triggering conventional inode-based monitoring alerts.

The per-pod MAF metric introduced in this framework directly exposes this dynamic. A steady MAF = 3 for a specific pod serves as a diagnostic signal indicating whiteout-heavy metadata amplification rather than direct inode consumption. When combined with the `container_inode_net_delta` metric, administrators gain visibility into both the magnitude and the nature of filesystem resource pressure (inode consumption and metadata amplification) simultaneously.

C. Event-Driven Tracing Closes the Polling Blind Spot

The framework’s event-driven architecture, built on the lock-free BPF ring buffer, provides a structural advantage over polling-based monitoring tools for high-frequency, short-lived filesystem activity. Conventional infrastructure monitoring tools sample host filesystem state at predetermined intervals, typically every 15 seconds. Workloads that create and destroy files within the gap between two polling intervals leaving no visible signature in node-level metrics such as inode usage or filesystem counters. The node’s free inode count and byte usage appear unchanged despite thousands of kernel operations having executed. The evasion workload in this study demonstrated this blind spot empirically: 2,000 create-delete cycles completed within ~15 seconds, leaving no visible signature in any rate-based Prometheus panel, while the framework’s cumulative `container_inode_net_delta` counter captured all 2,000 events with perfect accuracy.

This distinction between event-driven and state-polling approaches has implications beyond storage monitoring. Any kernel-level behavior that is transient by nature. Short-lived file handles, rapid metadata churn, burst-mode copy-up sequences

is structurally invisible to polling-based systems regardless of their sampling frequency. The eBPF ring buffer model, where every kernel event is pushed asynchronously to user space without sampling loss, is the only architectural approach that guarantees complete capture of kernel-level filesystem events.

D. Practical Implications for Cluster Administration

The per-pod `container_inode_net_delta` metric provides the direct diagnostic link that conventional tools cannot offer. Rather than observing a declining `node_inode_free` count and manually inspecting every running container to identify the responsible workload, an administrator can immediately identify the exact pod, namespace, and kernel mechanism responsible. The framework enables threshold-based alerting. For example, alerting when any pod accumulates a net filesystem activity delta (inode or metadata-based) exceeding a configurable threshold within a defined time window, transforming reactive node-outage scenarios into proactive, diagnosable events. The sub-11-millicore CPU overhead confirmed in evaluation indicates this level of observability is achievable without meaningful performance impact on production workloads.

V. CONCLUSION

This paper presented a dual-layer eBPF monitoring framework that simultaneously traces VFS-layer and OverlayFS semantic operations, maps events to Kubernetes pods via cgroup identity, and exports per-pod inode metrics through Prometheus. Controlled evaluation on a MicroK8s cluster produced two empirically quantified findings not previously reported in the literature. First, lower-layer file deletions produce a consistent **3:1** VFS metadata amplification factor associated with whiteout handling, while simultaneously producing near-neutral node-level inode counts, making this metadata amplification invisible to conventional inode-based monitoring. Second, copy-up operations exhibit a precise **1:1** inode consumption ratio per operation that remains constant regardless of container image layer depth. The second finding contradicts the common assumption that deeper image stacks proportionally amplify filesystem resource cost. Comparative evaluation confirmed that neither standard Kubernetes monitoring tools nor state-of-the-art eBPF security tracers provide equivalent per-pod semantic visibility, as both categories are structurally limited to observation points that cannot expose OverlayFS-internal kernel behavior.

Future work will extend the framework toward production-grade deployment through multi-node cluster aggregation via Prometheus federation, broader kernel version compatibility using BPF CO-RE portability, and integration of threshold-based alerting directly into the Prometheus metric pipeline for automated inode exhaustion prediction.